Estudiantes:
Axel Javier Rosas Rodríguez | A01738607
Elian Cantalapiedra Sabugal | A01738462
Edwin Emmanuel Salazar Meza | A01738380

Profesor:
Rigoberto Cerino Jiménez

Nombre de la Institución:
Instituto Tecnológico y de Estudios Superiores de
Tecnológico de Monterrey

Materia:
Implementación de robótica inteligente (Gpo 501)

Fecha:
17 de abril de 2025

Título del trabajo:
Actividad R_2. Cálculo del error de un robot móvil diferencial

Resumen

En esta actividad se implementó un sistema de localización para un robot móvil diferencial utilizando datos provenientes de los encoders de las ruedas. A partir de estos datos se estimaron las velocidades lineal y angular del robot, las cuales posteriormente se integraron mediante el método de Euler para obtener su posición en el plano cartesiano. Asimismo, se desarrolló un nodo adicional para calcular el error de distancia y el error angular respecto a un objetivo definido. La validación del sistema se realizó utilizando el nodo teleop_twist_keyboard, observando en tiempo real la evolución de la posición estimada, los errores calculados y la trayectoria seguida por el robot. Los resultados obtenidos confirman la correcta actualización de la localización y de los errores, evidenciando la base necesaria para un futuro control en lazo cerrado.

Objetivos

Objetivo General:

- Desarrollar e implementar una solución en ROS que permita la localización y navegación básica de un robot móvil diferencial tipo Puzzlebot, mediante el uso de odometría y el cálculo en tiempo real del error de posición y orientación hacia un objetivo deseado.

Objetivos específicos:

- Implementar un nodo en ROS para estimar la posición y orientación del robot a partir de los datos de los encoders.
- Verificar la estimación de la localización del robot mediante teleoperación utilizando teleop_twist_keyboard.
- Diseñar un nodo que calcule y publique los errores de distancia y orientación respecto a un punto objetivo.
- Analizar el comportamiento del robot durante la navegación punto a punto en lazo abierto.
- Validar que los ángulos calculados se mantengan dentro del rango de un círculo completo.

Introducción

La robótica móvil requiere que los robots sean capaces de conocer su posición dentro de un entorno y desplazarse de manera controlada hacia un objetivo específico. En este contexto, la localización y la navegación constituyen dos de los pilares fundamentales para el desarrollo de sistemas robóticos autónomos. En robots móviles diferenciales, una de las técnicas más empleadas para la estimación de la posición es la odometría, la cual utiliza la información proveniente de los encoders de las ruedas para calcular la posición y orientación del robot en el plano.

La odometría se basa en el modelo cinemático del robot diferencial, donde las velocidades lineales y angulares se integran a lo largo del tiempo para obtener las coordenadas (x,y) y el

ángulo de orientación θ. No obstante, esta técnica presenta limitaciones, principalmente la acumulación de errores, lo que hace indispensable su análisis y validación experimental.

Por otra parte, el cálculo del error de posición en un robot móvil diferencial permite determinar la distancia y el ángulo relativo entre la posición actual del robot y un objetivo deseado. Estos errores son fundamentales para evaluar el desempeño del sistema y constituyen la base para el desarrollo de estrategias de control y navegación más avanzadas.

La odometría en un robot móvil diferencial es el proceso mediante el cual se estima su posición y orientación en el plano a partir de las mediciones de movimiento de sus ruedas. Este método se basa en el modelo cinemático del robot, el cual relaciona las velocidades de las ruedas izquierda y derecha con la velocidad lineal y angular del sistema. A partir de estas velocidades, la posición del robot se obtiene integrando las ecuaciones de movimiento a lo largo del tiempo. Aunque la odometría es computacionalmente eficiente y ampliamente utilizada en robótica móvil, presenta limitaciones importantes, como la acumulación de error debido al deslizamiento de las ruedas, imprecisiones en los encoders y aproximaciones numéricas.

El concepto de Quality of Service (QoS) en ROS 2 define las políticas de comunicación entre nodos, determinando cómo se transmiten y reciben los mensajes en el sistema. Los parámetros de QoS permiten configurar aspectos como la confiabilidad, la durabilidad y la profundidad de la cola de mensajes. En aplicaciones de robótica móvil, como la odometría, una configuración adecuada de QoS es esencial para garantizar la recepción continua y en tiempo real de la información de estado del robot. Una mala configuración puede provocar pérdida de mensajes o retrasos, afectando directamente la precisión de la localización y el cálculo del error. (Quality of Service Settings — ROS 2 Documentation: Rolling Documentation, 2025)

En este mini reto se emplea ROS como plataforma de desarrollo, aprovechando sus capacidades de comunicación entre nodos mediante mensajes estandarizados, como los mensajes de odometría. La integración de estos conceptos nos permite entender sus conocimientos en localización, navegación punto a punto y análisis del comportamiento de robots móviles.

Metodología

- Se configuró un workspace en ROS 2 para el desarrollo de los nodos de estimación y cálculo de error.
- Se implementó un nodo para convertir los datos de los encoders en velocidad lineal y angular del robot móvil diferencial.
- Se publicaron las velocidades estimadas utilizando el mensaje estándar nav_msgs/Odometry.
- Se integraron las velocidades mediante el método de Euler para estimar la posición (x,y) y la orientación (θ) del robot.
- Se transformó la orientación del robot a cuaterniones, cumpliendo con el estándar de orientación de ROS 2.

- Se validó la localización del robot mediante el uso de teleop_twist_keyboard, comparando comandos y estimaciones.
- Se definió un objetivo en el plano cartesiano y se desarrolló un nodo para el cálculo del error de distancia y error angular.
- Se normaliza el error angular para mantenerlo dentro del rango [−π,π].
- Se verificó el funcionamiento del sistema observando en tiempo real la actualización del error y la trayectoria del robot.

Solución del problema

Actividad 1

```python
# Callbacks
def encR_callback(self, msg):
    self.wr = msg
    self.get_logger().info(f'Vel Derecha: {self.wr.data}')

def encL_callback(self, msg):
    self.wl = msg
    self.get_logger().info(f'Vel Izquierda: {self.wl.data}')
```

Imagen 1. Lectura de las Velocidades en los Encoders

Este nodo implementa la estimación de la odometría de un robot diferencial a partir de las velocidades de sus ruedas. Primero, se suscribe a los tópicos VelocityEncR y VelocityEncL, donde recibe las velocidades angulares de las ruedas derecha e izquierda. Con esta información se calculan las velocidades tangenciales de cada rueda y posteriormente se obtiene la velocidad lineal (V) y angular (Omega) del robot mediante su modelo cinemático.

```python
# Get current time and compute dt
current_time = self.get_clock().now()
dt = (current_time - self.last_time).nanoseconds * 1e-9  # Convert to seconds

if dt > self._sample_time:
    #Wheel Tangential Velocities
    self.v_r = self._r * self.wr.data
    self.v_l = self._r * self.wl.data

    #Robot Velocities
    self.V = (0.5) * (self.v_r + self.v_l)
    self.Omega = (1.0 / self._l) * (self.v_r - self.v_l)

    self.Th += self.Omega * dt
    self.Th = np.arctan2(np.sin(self.Th), np.cos(self.Th))
    self.X += self.V * np.cos(self.Th) * dt
    self.Y += self.V * np.sin(self.Th) * dt

    self.last_time = current_time

    self.publish_odometry()
    self.get_logger().info(f'Vel_L:{self.V}, Vel_A:{self.Omega}')
    self.get_logger().info(f'X:{self.X} Y:{self.Y} tetha:{self.Th}')
```

Imagen 2. Cálculo de la velocidad Lineal y Angular

Actividad 2

Para poder calcular la posición del PuzzleBot, se utiliza el método de integración de Euler que consiste en aproximar el nuevo estado del sistema sumando al estado actual el cambio producido por sus velocidades durante ese intervalo de tiempo.

```python
self.Th += self.Omega * dt
self.Th = np.arctan2(np.sin(self.Th), np.cos(self.Th))
self.X += self.V * np.cos(self.Th) * dt
self.Y += self.V * np.sin(self.Th) * dt
```

Imagen 3. Cálculo de la Posición Angular, X y Y

En el caso de la orientación, el ángulo del robot se actualiza sumando el producto de la velocidad angular por dt, lo que representa cuánto ha girado el robot en ese lapso. Posteriormente, este ángulo se normaliza para mantenerlo dentro de un rango válido y evitar discontinuidades.

```python
def publish_odometry(self):
    """ Publishes odometry message with updated state """
    q1 = transforms3d.euler.euler2quat(0, 0, self.Th)

    self.odom_msg.header.stamp = self.get_clock().now().to_msg()
    self.odom_msg.header.frame_id = 'odom'
    self.odom_msg.child_frame_id = "base_footprint"

    # Posición
    self.odom_msg.pose.pose.position.x = self.X
    self.odom_msg.pose.pose.position.y = self.Y
    self.odom_msg.pose.pose.position.z = 0.0

    # Orientación
    self.odom_msg.pose.pose.orientation.x = q1[1]
    self.odom_msg.pose.pose.orientation.y = q1[2]
    self.odom_msg.pose.pose.orientation.z = q1[3]
    self.odom_msg.pose.pose.orientation.w = q1[0]

    # Velocidades
    self.odom_msg.twist.twist.linear.x = self.V
    self.odom_msg.twist.twist.angular.z = self.Omega

    self.odom_pub.publish(self.odom_msg)
```

Imagen 4. Información enviada por un mensaje de tipo Odometry

Actividad 3

Este último nodo se encarga de calcular el error que existe entre la posición actual del robot y la posición objetivo. Para ello, se suscribe al tópico odom, del cual obtiene la posición (x,y)

y la orientación del robot. También se suscribe al tópico target, donde recibe la posición deseada (x_d, y_d, θ_d).

```python
# Callbacks
def odom_callback(self, msg):
    self.x = msg.pose.pose.position.x
    self.y = msg.pose.pose.position.y
    q = msg.pose.pose.orientation
    _, _, self.th = transforms3d.euler.quat2euler([q.w, q.x, q.y, q.z])

def target_callback(self, msg):
    self.x_d = msg.x
    self.y_d = msg.y
    self.th_d = msg.z
```

Imagen 5. Suscripción a los tópicos de /odom y /target

A partir de esta información, en cada iteración del temporizador se calcula el error como la diferencia entre la posición deseada y la actual. Este error se empaqueta en un mensaje de tipo Vector3 y se publica en el tópico error, además de imprimir estos errores en la consola.

```python
def run(self):
    ex = self.x_d - self.x
    ey = self.y_d - self.y
    eth = self.th_d - self.th
    #eth = np.arctan2(np.sin(eth), np.cos(eth))

    msg = Vector3()
    msg.x = ex
    msg.y = ey
    msg.z = eth
```

Imagen 6. Cálculo del error

Resultados

Actividad 1

¿Qué pasa si una rueda gira más rápido?

Internamente el PuzzleBot ya cuenta con control interno para garantizar una velocidad lineal y angular del robot. Si una rueda gira más rápido que la otra, el robot genera una velocidad angular o lineal ligeramente distinta a la establecida, y a pesar del control interno el PuzzleBot, al manejarse en lazo abierto el robot se mueve ligeramente a otra dirección.

¿Qué pasa si ambas giran igual?

Si ambas ruedas giraran a la misma velocidad, la velocidad angular sería, por lo que el robot se movería en una línea completamente recta, con una velocidad constante sin cambiar su orientación.

Comparación cmd_vel vs velocidad estimada

Primero definimos una velocidad a usar en Teleop, el nodo de cmd_vel recibe directamente este parámetro según la tecla que presionemos, en este ejemplo estamos utilizando solo la velocidad angular de 1.0 rad/s que es recibida por cmd_vel.

```
currently:      speed 0.13840459435251415 turn 1.0
currently:      speed 0.15224505378776557 turn 1.0
currently:      speed 0.16746955916654213 turn 1.0
currently:      speed 0.18421651508319636 turn 1.0
```

Imagen 7. Velocidad Lineal y Angular definida con Teleop

Al usar nuestro programa que calcula las velocidades, podemos notar que la velocidad angular nunca llega a ser igual 1.0 rad/s, esta varía ligeramente, teniendo aproximadamente un error de 0.1 rad/s. Esto se debe a que nuestras ruedas no giran a la misma velocidad, el escenario ideal es que giren en sentido contrario una respecto a la otra, de modo que la velocidad lineal sea 0 m/s.

Pero al no cumplirse esta condición, debido a que la rueda izquierda gira ligeramente más rápido, se presenta una velocidad lineal que desplaza al robot ligeramente conforme avanza el tiempo.

```
X:-0.003409575298722714 Y:0.012682447520200093 tetha:-0.7789468256554521
Vel Derecha: -1.7767822742462158
Vel Izquierda: 1.7766456604003906
Vel Derecha: -1.5361559391021729
Vel Izquierda: 1.537229299545288
Vel_L:2.6834011077885023e-05, Vel_A:-0.903936834896312
X:-0.0034091608661951872 Y:0.012682021688450226 tetha:-0.7989635658659857
Vel Derecha: -1.7688096761703491
Vel Izquierda: 2.1212072372436523
Vel Derecha: -2.059290647506714
Vel_L:0.0015479147434234591, Vel_A:-1.2295582013971664
X:-0.0033903970897085256 Y:0.012661878788215137 tetha:-0.8208302969633207
Vel Izquierda: 1.764971375465393
Vel Derecha: -1.5360027551651
Vel Izquierda: 1.5370759963989258
Vel Derecha: -2.102862596511841
Vel_L:-0.014144665002822875, Vel_A:-1.0705701743855194
X:-0.0035924746654749902 Y:0.012889074106732394 tetha:-0.8438438204932709
Vel Izquierda: 2.1030540466308594
Vel Derecha: -1.776918888092041
Vel Izquierda: 2.0729126930236816
Vel_L:0.007399845123291018, Vel_A:-1.1323034062105066
```

Imagen 8. Velocidad Angular de 1.0 rad/s y Velocidad Lineal de 0 m/s

Al graficar la trayectoría, el punto rojo nos representa la posición actual de PuzzleBot, mientras que el punto verde la posición original de x = 0, y = 0. Al contrastar la gráfica con el análisis de las velocidades, vemos que efectivamente el PuzzleBot se desplaza cuando lo ideal es que siempre se mantuviera en (0,0).

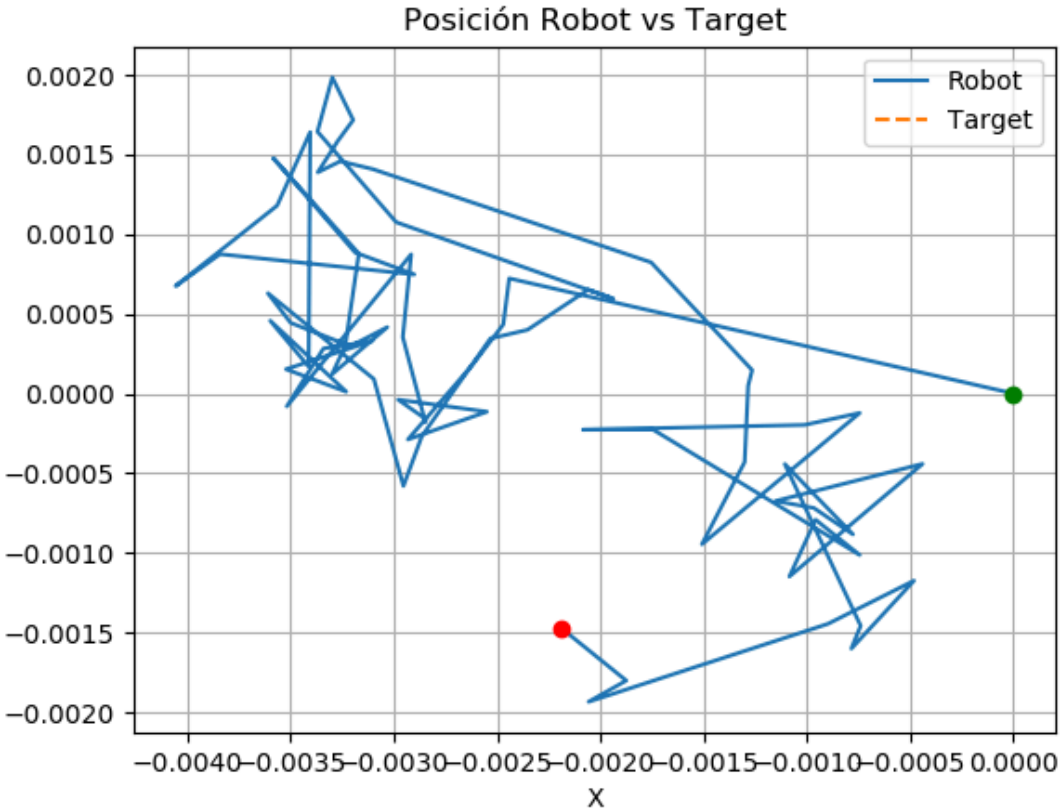

Imagen 9. Trayectoria del Robot girando sobre su propio eje

Aun así, el desplazamiento presentado es mínimo, pero conforme pasa el tiempo el error se acumula aumentando el error.

Actividad 2

Para esta segunda actividad analizamos dos escenarios, uno donde el robot gira a una velocidad lineal de 0.18 m/s y una velocidad angular de 1.0 rad/s de modo que grafique una elipse.

Podemos observar que el robot sigue perfectamente esta trayectoria, sin embargo al encontrarse en un control de lazo abierto, conforme avanza el tiempo el error se acumula, desviándose ligeramente tras cada iteración.

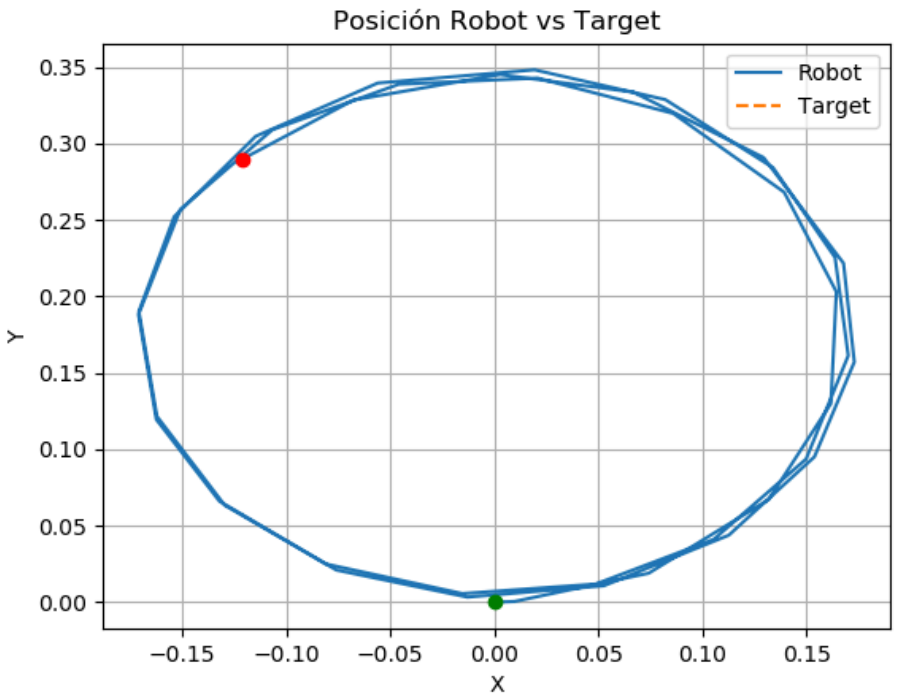

Imagen 10. Trayectoria del Robot - Escenario 1

En el segundo escenario se controló el PuzzleBot para que recorriera 2 metros a una velocidad 0.18 m/s y regresara a su punto inicial. Al hacer este movimiento con Teleop, se obtuvo la siguiente gráfica.

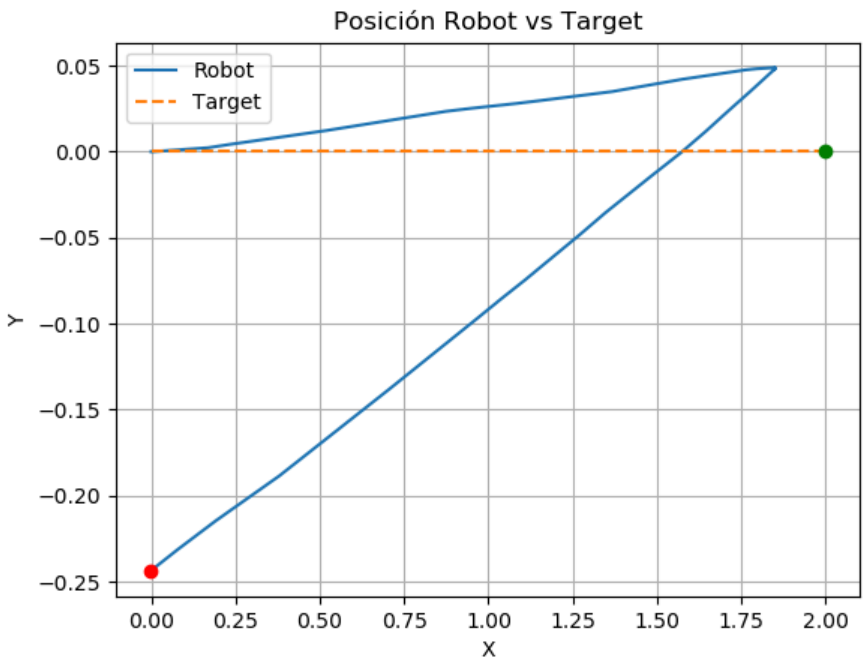

Imagen 11. Trayectoria del Robot - Escenario 2

Como puede observarse, aunque físicamente el robot ya había recorrido 2 metros, la estimación de la posición nos dice que apenas había alcanzado 1 metro con 80 centímetros,

esto se debe a que el cálculo de la posición no considera otros efectos como lo son el deslizamiento.

¿La trayectoria es uniforme o se desvía?

Se desvía ligeramente, especialmente cuando se realiza un giro, pues las velocidades desiguales en los encoders y que el robot no alcanza la velocidad enviada por cmd_vel, provoca que se acumule el error desviandose.

Si se desvía, ¿Por qué se desvía?

Primero, porque no se tiene un control en lazo cerrado, el control el lazo cerrado ayudaría a mitigar las desviaciones ocasionadas por errores en el cálculo de la velocidad, sin embargo, falta considerar muchos otros factores como es el deslizamiento que permitan al PuzzleBot seguir una trayectoria definida.

Actividad 3

Analizando los errores presentes en el segundo escenario, lo dividimos en dos partes, la primera el error de cuando llega a la posición objetivo, y la segunda con el error cuando regresa a la posición original.

```
E_x: 0.23763137646754506, E_y: -0.04754833717540165 5, E_th: -0.0069514858022266595
E_x: 0.23763137646754506, E_y: -0.04754833717540165 5, E_th: -0.0069514858022266595
E_x: 0.23763137646754506, E_y: -0.04754833717540165 5, E_th: -0.0069514858022266595
```

Imagen 12. Errores PuzzleBot cuando llega la meta.

Como puede apreciarse en estos primeros resultados, el PuzzleBot logra seguir el camino hacia la meta, aunque calculando un poco mal su posición debido a la existencia de deslizamiento en las ruedas, este error disminuye al cambiar la superficie por la cual se desplaza el PuzzleBot.

```
E_x: 2.004263821846905, E_y: 0.24409888033395097, E_th: 3.012734632323383
E_x: 2.004263821846905, E_y: 0.24409888033395097, E_th: 3.012734632323383
E_x: 2.004263821846905, E_y: 0.24409888033395097, E_th: 3.012734632323383
```

Imagen 13. Errores PuzzleBot cuando llega al origen

Por otra parte cuando hace el camino de regreso, las ruedas presentan una desviación ligera en el ángulo, provocando que aunque recorra la misma distancia en X, lo termina situando en otra posición en Y.

Conclusiones

En la actividad se logró implementar de manera satisfactoria un sistema de localización y cálculo del error para un robot móvil diferencial, empleando odometría basada en encoders dentro de ROS 2. A partir de la estimación de las velocidades lineal y angular del robot y su posterior integración mediante el método de Euler, fue posible obtener una estimación continua de la posición y orientación del robot en el plano cartesiano. Asimismo, se

desarrolló un nodo específico para el cálculo del error de distancia y del error angular respecto a un objetivo definido, verificando que dichos valores se actualizan correctamente durante la operación del robot.

Los objetivos planteados se cumplieron, ya que se validó experimentalmente la estimación de la localización y el comportamiento del robot durante la navegación punto a punto en lazo abierto utilizando teleoperación. Las pruebas realizadas permitieron observar fenómenos reales del sistema, como ligeras desviaciones en la trayectoria y diferencias entre las velocidades comandadas y las velocidades estimadas, las cuales se atribuyen a las limitaciones propias de la odometría, tales como imprecisiones en los encoders, diferencias en el giro de las ruedas y la ausencia de retroalimentación externa.

A pesar de estas limitaciones, los resultados obtenidos demuestran que la estimación de la localización y el cálculo del error son consistentes y adecuados como base para el desarrollo de estrategias de control más avanzadas. Como trabajo futuro, se propone integrar técnicas de control en lazo cerrado y sensores adicionales que permitan corregir los errores acumulados, mejorar la precisión de la navegación y avanzar hacia un sistema de navegación autónoma más robusto.

Referencias

Odometría. (n.d.). Game Manual 0. https://gm0.org/es/latest/docs/software/concepts/odometry.html

Quality of Service settings — ROS 2 Documentation: Rolling documentation. (2025). Ros.Org. https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Quality-of-Service-Settings.html